

Recruitment Management System

Python Application Using Modules and File Handling

Solution Overview

This console-based Python application manages job postings, candidates, applications, recruitment status, reports, validation, and activity logging. It stores jobs and applications in JSON files and candidate records in a CSV file.

Requirement	Implementation
Classes	Job, Candidate, and Application dataclasses
File handling	jobs.json, candidates.csv, and applications.json
Workflow	Create application and update its hiring status
Reports	Applications per job and shortlisted candidates
Validation	Duplicate IDs/applications and missing job or candidate IDs
Audit trail	application_activity.log records application actions

How to Run

Save the program as **recruitment_system.py** and run `python recruitment_system.py`. The program automatically creates a data folder and its required storage files when records are added.

Complete Python Program

```
# recruitment_system.py
import json
import csv
import logging
from dataclasses import dataclass, asdict
from pathlib import Path

DATA_DIR = Path("data")
DATA_DIR.mkdir(exist_ok=True)
JOBS_FILE = DATA_DIR / "jobs.json"
CANDIDATES_FILE = DATA_DIR / "candidates.csv"
APPLICATIONS_FILE = DATA_DIR / "applications.json"

logging.basicConfig(filename="application_activity.log", level=logging.INFO,
                    format="%(asctime)s - %(message)s")

@dataclass
class Job:
    job_id: str
    title: str
    company: str
    industry: str
    location: str
    status: str = "Open"

@dataclass
class Candidate:
    candidate_id: str
    name: str
    email: str
    skills: str

@dataclass
class Application:
    application_id: str
    job_id: str
    candidate_id: str
    status: str = "Applied"

def load_json(file_path):
    if not file_path.exists():
        return []
    with open(file_path, "r", encoding="utf-8") as file:
        return json.load(file)

def save_json(file_path, data):
    with open(file_path, "w", encoding="utf-8") as file:
        json.dump(data, file, indent=4)

def load_candidates():
    if not CANDIDATES_FILE.exists():
        return []
    with open(CANDIDATES_FILE, "r", newline="", encoding="utf-8") as file:
        return list(csv.DictReader(file))

def save_candidates(candidates):
    with open(CANDIDATES_FILE, "w", newline="", encoding="utf-8") as file:
        fields = ["candidate_id", "name", "email", "skills"]
        writer = csv.DictWriter(file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(candidates)

def add_job(job):
    jobs = load_json(JOBS_FILE)
    if any(item["job_id"] == job.job_id for item in jobs):
        print("Error: Job ID already exists.")
        return
    jobs.append(asdict(job))
    save_json(JOBS_FILE, jobs)
    print("Job added successfully.")

def add_candidate(candidate):
    candidates = load_candidates()
    if any(item["candidate_id"] == candidate.candidate_id for item in candidates):
        print("Error: Candidate ID already exists.")
        return
    candidates.append(asdict(candidate))
    save_candidates(candidates)
    print("Candidate added successfully.")

def apply_for_job(application):
    jobs = load_json(JOBS_FILE)
    candidates = load_candidates()
    applications = load_json(APPLICATIONS_FILE)

    if not any(job["job_id"] == application.job_id for job in jobs):
```

```

        print("Error: Job ID does not exist.")
        return
    if not any(c["candidate_id"] == application.candidate_id for c in candidates):
        print("Error: Candidate ID does not exist.")
        return
    if any(app["job_id"] == application.job_id and
           app["candidate_id"] == application.candidate_id for app in applications):
        print("Error: Candidate has already applied for this job.")
        return

    applications.append(asdict(application))
    save_json(APPLICATIONS_FILE, applications)
    logging.info(f"Application created: {application.candidate_id} applied for {application.job_id}")
    print("Application submitted successfully.")

def update_application_status(application_id, new_status):
    valid_statuses = ["Applied", "Under Review", "Shortlisted", "Rejected", "Hired"]
    applications = load_json(APPLICATIONS_FILE)
    if new_status not in valid_statuses:
        print("Error: Invalid status.")
        return
    for app in applications:
        if app["application_id"] == application_id:
            app["status"] = new_status
            save_json(APPLICATIONS_FILE, applications)
            logging.info(f"Application {application_id} status changed to {new_status}")
            print("Application status updated.")
            return
    print("Error: Application ID not found.")

def applications_per_job_report():
    jobs = load_json(JOBS_FILE)
    applications = load_json(APPLICATIONS_FILE)
    print("\n--- Applications per Job Report ---")
    for job in jobs:
        total = sum(1 for app in applications if app["job_id"] == job["job_id"])
        print(f"{job['title']} ({job['job_id']}: {total} application(s)")

def shortlisted_candidates_report():
    candidates = load_candidates()
    applications = load_json(APPLICATIONS_FILE)
    shortlisted = {app["candidate_id"] for app in applications if app["status"] == "Shortlisted"}
    print("\n--- Shortlisted Candidates ---")
    for candidate in candidates:
        if candidate["candidate_id"] in shortlisted:
            print(f"{candidate['name']} - {candidate['email']}")

def main():
    while True:
        print("\n1. Add Job\n2. Add Candidate\n3. Apply for Job")
        print("4. Update Application Status\n5. Applications per Job Report")
        print("6. Shortlisted Candidates Report\n7. Exit")
        choice = input("Enter your choice: ")
        if choice == "1":
            add_job(Job(input("Job ID: "), input("Title: "), input("Company: "),
                          input("Industry: "), input("Location: ")))
        elif choice == "2":
            add_candidate(Candidate(input("Candidate ID: "), input("Name: "),
                                     input("Email: "), input("Skills: ")))
        elif choice == "3":
            apply_for_job(Application(input("Application ID: "), input("Job ID: "),
                                     input("Candidate ID: ")))
        elif choice == "4":
            update_application_status(input("Application ID: "), input("New Status: "))
        elif choice == "5":
            applications_per_job_report()
        elif choice == "6":
            shortlisted_candidates_report()
        elif choice == "7":
            break
        else:
            print("Invalid choice. Please try again.")

if __name__ == "__main__":
    main()

```